

INTERPOLATION

1. Newton / Lagrange

Polynomial through values at distinct nodes.

- Each node listed once; degree $n = (\# \text{nodes}) - 1$.
- Divided-difference table: $f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}$.
- Coefficients = top entry of each column.
- Newton form: $N(x) = c_0 + c_1(x - x_0) + \dots + c_n \prod_{i=0}^{n-1} (x - x_i)$.
- Lagrange form: $L(x) = \sum_i f(x_i) \ell_i(x)$, $\ell_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$.
- $f \in C^{n+1}$ remainder: $R(x) = \frac{\prod_i (x - x_i)}{(n+1)!} f^{(n+1)}(\xi)$.

Table (3 nodes) — coefficients = top of each column: f_0 , $f[x_0, x_1]$, $f[x_0, x_1, x_2]$.

x0		f0		
			f[x0,x1]	
x1		f1		f[x0,x1,x2]
			f[x1,x2]	
x2		f2		

2. Hermite

Polynomial through values and consecutive derivatives at the nodes.

- Each node repeated by multiplicity (value only $\rightarrow 1$, value + $f' \rightarrow 2$, ...); degree $n = \sum_i (r_i + 1) - 1$.
- Divided-difference table; first column = f -values, repeated per multiplicity.
- All-same-node brackets seeded by derivatives: $f[\underbrace{x, \dots, x}_{k+1}] = \frac{f^{(k)}(x)}{k!}$. Mixed brackets use the ordinary quotient.
- Coefficients = top entry of each column.
- Newton form; products follow the listed (repeated) node order.
- $f \in C^{n+1}$ remainder: $R(x) = \frac{u(x)}{(n+1)!} f^{(n+1)}(\xi)$, $u(x) = \prod_i (x - x_i)^{r_i+1}$.

Table — double node x_0 (repeated) + simple x_1 ; seed the all- x_0 bracket $f[x_0, x_0] = f'(x_0)$. Coeffs: f_0 , f'_0 , $f[x_0, x_0, x_1]$.

x0		f0		
			f'0	<- seed = f'(x0)
x0		f0		f[x0,x0,x1]
			f[x0,x1]	
x1		f1		

LEAST-SQUARES APPROXIMATION

3. Least squares fit

Polynomial $\varphi(x) = \sum_{j=1}^{k+1} c_j \pi_j(x)$ of degree k best-fitting $N > k + 1$ points (x_i, y_i) .

- Monomial basis $\pi_1 = 1$, $\pi_2 = x$, ..., $\pi_{k+1} = x^k$.
- Power sums $S_p = \sum_i x_i^p$ ($p = 0, \dots, 2k$) and $T_p = \sum_i x_i^p y_i$ ($p = 0, \dots, k$).
- Normal equations $A\mathbf{c} = \mathbf{b}$ with $a_{ij} = S_{i+j-2}$, $b_i = T_{i-1}$ — symmetric positive definite.
- Coefficients $\mathbf{c}$ from the $(k+1) \times (k+1)$ system.
- φ^* does not pass through the data; residuals sum to 0.

NUMERICAL INTEGRATION (QUADRATURE)

4. Weighted 1-point quadrature

$\int_a^b w(x)f(x) dx = A f(\alpha) + R(f)$ — maximum degree of precision, $f \in C^2$.

- Max DoP for $n = 1$ is $2n - 1 = 1$.
- Moments $\mu_k = \int_a^b w(x) x^k dx$ for $k = 0, 1, 2$.
- Exactness: $f = 1 \Rightarrow A = \mu_0$; $f = x \Rightarrow \alpha = \mu_1/\mu_0$.
- DoP = 1, since $f = x^2$ already breaks exactness.
- Remainder via Peano ($d = 1$, $f \in C^2$): functional $R(g) = \int_a^b w g dx - A g(\alpha)$, kernel

$$K(t) = R_x[(x - t)_+] = \int_t^b w(x)(x - t) dx - A(\alpha - t)_+.$$

The $(\alpha - t)_+$ piece comes from the node term $A f(\alpha)$ and switches off for $t > \alpha$ (so K is piecewise, splitting at $t = \alpha$).

- Check $K(t)$ keeps its sign on $[a, b] \rightarrow$ the MVT pulls f'' out: $R(f) = f''(\xi) \int_a^b K(t) dt$.
- Constant via the shortcut $\int_a^b K = \frac{R(x^2)}{2!}$ (no need to integrate K): $R(f) = \frac{R(x^2)}{2!} f''(\xi) = \frac{\mu_2 - A\alpha^2}{2} f''(\xi)$, $\xi \in (a, b)$, with $R(x^2) = \mu_2 - A\alpha^2$.

5. Quadrature from interpolation + Peano

Quadrature obtained by integrating an interpolating polynomial; degree of precision and Peano remainder.

- Interpolating polynomial Pf from the given data; degree = $(\# \text{conditions}) - 1$.
- Quadrature formula: $\int_a^b (Pf)(x) dx$ — a linear combination of the data samples with their basis integrals as weights.
- Degree of precision d : the first monomial $1, x, x^2, \dots$ that breaks exactness fixes d .
- Peano kernel $K(t) = R_x\left[\frac{(x - t)_+^d}{d!}\right]$. For functional terms with a derivative $f'(x_0)$, $\frac{d}{dx}(x - t)_+ = H(x - t)$ (unit step at t).
- Kernel must keep its sign on $[a, b]$.
- $f \in C^{d+1}$ remainder: $R(f) = \frac{R(x^{d+1})}{(d+1)!} f^{(d+1)}(\xi)$, $\xi \in (a, b)$, with $R(x^{d+1}) = \int_a^b x^{d+1} dx -$ (quadrature on x^{d+1}).

NONLINEAR ITERATION

6. Fixed-point convergence & order

Iteration $x_{n+1} = g(x_n)$ toward a fixed point $g(\alpha) = \alpha$ (a root of $x - g(x)$).

- Local convergence: $|g'(\alpha)| < 1$ for x_0 close; an interval needs $g([a, b]) \subseteq [a, b]$ and $\max_{[a,b]} |g'| < 1$.
- Order p = index of first nonzero derivative of g at α ; constant $\frac{g^{(p)}(\alpha)}{p!}$.
- $g'(\alpha) \neq 0 \rightarrow$ linear (rate $|g'(\alpha)|$); $g'(\alpha) = 0$, $g''(\alpha) \neq 0 \rightarrow$ quadratic; etc.
- Parametrized $g(x; c)$: the c solving $g'(\alpha; c) = 0$ lifts the order $1 \rightarrow \geq 2$; convergence range from $|g'(\alpha; c)| < 1$.
- Errors: linear $|e_n| \leq \lambda^n |e_0|$ ($\lambda = \max_I |g'|$); order p : $|e_{n+1}| \leq \frac{M_p}{p!} |e_n|^p$.

7. Banach contraction (existence/uniqueness)

Proves $x = g(x)$ has a unique solution and the iteration converges.

- g is a contraction on $[a, b]$ if $g([a, b]) \subseteq [a, b]$ and $|g'(x)| \leq \lambda < 1$ (then $|g(x) - g(y)| \leq \lambda|x - y|$ by MVT).
- Banach principle: unique fixed point $\alpha \in [a, b]$; converges from any $x_0 \in [a, b]$.
- A priori: $|x_n - \alpha| \leq \lambda^n |x_0 - \alpha| \leq \frac{\lambda^n}{1 - \lambda} |x_1 - x_0|$.
- A posteriori: $|x_n - \alpha| \leq \frac{\lambda}{1 - \lambda} |x_n - x_{n-1}|$.

8. Newton's method (rootfinding)

Solve $f(x) = 0$: $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} = g(x_n)$.

- Simple root ($f(\alpha) = 0$, $f'(\alpha) \neq 0$): $g'(\alpha) = 0 \rightarrow$ quadratic, $\lim \frac{e_{n+1}}{e_n^2} = \frac{f''(\alpha)}{2f'(\alpha)}$.
- Square root $\sqrt{a}$: $f = x^2 - a \rightarrow g(x) = \frac{1}{2}(x + \frac{a}{x})$.
- Error: $|e_{n+1}| \leq \frac{M}{2m} |e_n|^2$, $M = \max_I |f''|$, $m = \min_I |f'|$.
- Multiple root (mult. m): only linear, $g'(\alpha) = 1 - \frac{1}{m}$; restore quadratic via $x_{n+1} = x_n - m \frac{f(x_n)}{f'(x_n)}$.

LINEAR SYSTEMS $AX = B$

9. Gaussian elimination

Direct solve by reducing the augmented matrix to upper-triangular form, then back-substituting.

- Work on the augmented matrix $[A | b]$.
- Forward elimination, column by column: multiplier $m_{ij} = a_{ij}/a_{jj}$ ($i > j$), then $R_i \leftarrow R_i - m_{ij}R_j$, until upper-triangular.
- Back-substitution from the last row up: $x_i = \frac{1}{a_{ii}} (b_i - \sum_{j>i} a_{ij}x_j)$.
- Zero pivot $\rightarrow$ swap in a lower row (partial pivoting $\rightarrow$ LUP).

10. LUP factorization

$PA = LU$ for general nonsingular A ; the system reduces to two triangular substitutions.

- Stage j : the largest- $|\cdot|$ entry in column j at/below the diagonal becomes pivot; the row swap goes into P . **Multipliers stored in earlier columns ride along with their rows.**
- Multipliers $m_{ij} = a_{ij}/a_{jj}$ for $i > j$, stored in L .
- Elimination $R_i \leftarrow R_i - m_{ij}R_j$ for $i > j$.
- Stages run $j = 1, \dots, n-1$.
- L unit lower (current row order), U upper, P the cumulative permutation.
- Forward $Ly = Pb$, then back $Ux = y$.

11. Cholesky factorization

SPD factorization $A = R^T R$; the system reduces to two triangular substitutions.

- SPD test: all leading principal minors > 0 .
- Symmetric elimination $\ell_{ij} = a_{ij}^{(j-1)}/a_{jj}^{(j-1)}$ for $i > j$, then $R_i \leftarrow R_i - \ell_{ij}R_j$. Multipliers ℓ_{ij} in the unit-lower L ; emerging pivots d_j form $D = \text{diag}(d_1, \dots, d_n)$, so $A = LDL^T$.
- $R = \sqrt{D}L^T$, equivalently $R^T = L\sqrt{D}$, giving $A = R^T R$.
- Forward substitution $R^T y = b$ (top $\rightarrow$ down).
- Back substitution $Rx = y$ (bottom $\rightarrow$ up).

If any $d_j \leq 0$, A is not SPD — fall back to LUP.

12. Jacobi

Iteration for $Ax = b$ using only the previous iterate.

- Strict diagonal dominance $\Rightarrow$ convergence.
- Component formula:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left[b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right].$$

Fresh components are **not** reused within the same sweep.

- Iterate from $x^{(0)}$.

Matrix form: $x^{(k+1)} = D^{-1}(L + U)x^{(k)} + D^{-1}b$.

13. Gauss-Seidel

Iteration for $Ax = b$ — reuses fresh components within each sweep.

- Strict diagonal dominance $|a_{ii}| > \sum_{j \neq i} |a_{ij}| \Rightarrow$ convergence.
- Component formula:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left[b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right].$$

Just-computed $x_j^{(k+1)}$ used for $j < i$.

- Sweep $i = 1, \dots, n$ from $x^{(0)}$.

Matrix form: $x^{(k+1)} = (D - L)^{-1}Ux^{(k)} + (D - L)^{-1}b$.